

클라우드 구축 지침서

Google Cloud에 무엇을 어떻게 올릴 것인가

파일 하나하나의 목적지 · 버킷 구조 · DB 스키마 ·
적재 파이프라인 · 배포 · 권한까지

프로젝트와 버킷은 이미 만들어져 있습니다. 이 문서는 그 위에서 실제로 무엇을 만들고
어떤 데이터를 어디에 넣을지를 명령어 수준까지 정리한 작업 지침서입니다.
프로젝트 배경·하드웨어·역할 설명은 앞선 자료에서 다루었으므로 반복하지 않습니다.

PROJECT	hanium-ssu-kpc		
REGION	asia-northeast3	(서울)	
BUCKET	gs://hanium-v2x	Standard	· 비공개

AI 기반 V2X 협력형 지능형 시각장애인 보행 안전 지팡이 시스템

팀 하이퍼-송 · 2026 ICT 한이음 드림업 프로젝트 · 2026. 7.

0 이 문서의 범위와 현재 상태

클라우드 구축에만 집중합니다. 실제 작업할 때는 9장을 펼쳐 놓고 따라 하세요.

1. 목표 아키텍처와 리소스 목록
2. 데이터 파일 카탈로그 ★
3. Cloud Storage 설계
4. Cloud SQL 설계와 스키마
5. 지도 3종 데이터 모델
6. 적재 파이프라인 (실시간·배치)
7. Cloud Run 배포
8. IAM · 시크릿 · 보안
9. 실행 순서 (콘솔 + gcloud)
10. 운영 — 비용 · 백업 · 모니터링
11. 완료 기준 체크리스트
12. 자주 막히는 곳과 해결
13. 용어집
14. 지금 팀이 결정해야 할 것

0.1 이미 확보된 것

항목	값	비고
프로젝트	hanium-ssu-kpc	모든 리소스가 여기에 들어갑니다
버킷	gs://hanium-v2x	Region · Standard · 2026-07-24 생성
버킷 위치	asia-northeast3	서울. 앞으로 만들 리소스도 전부 이 리전
공개 액세스	공개 아님	올바른 상태입니다. 바꾸지 마세요

0.2 앞으로 만들 것

버킷 안의 폴더 구조, Cloud SQL 인스턴스와 테이블, Artifact Registry, Cloud Run 서비스, 서비스 계정과 시크릿. 이 문서가 그 전부를 다룹니다.

클라우드에는 Google Cloud 하나만 씁니다

다른 클라우드(AWS 등)와 나눠 쓰지 않기로 확정했습니다. 따라서 이 문서의 모든 리소스는 hanium-ssu-kpc 프로젝트 안, asia-northeast3 리전에 만듭니다.

이렇게 통일하면 **권한 체계가 하나(IAM)**, **비용 청구서가 하나**, **리전 간 전송 요금 없음**이라는 이점이 있습니다. 나중에 다른 클라우드를 검토하게 되더라도, 지금 단계에서 나누는 것은 얻는 것보다 잃는 것이 많습니다.

전제 — 서버는 안전 기능이 아닙니다

실시간 경고(지팡이 진동)는 현장에서 끝나며 **서버를 왕복하지 않습니다**. 클라우드가 통째로 멈춰도 보행자 안전 기능은 정상 작동합니다. 따라서 **가용성보다 비용과 단순함을 우선**해 설계했습니다.고가용성(HA), 다중 리전, 로드밸런서를 넣지 않은 이유가 이것입니다.

이 문서의 표기

[팀 확인 필요] — 혼자 정하면 안 되는 항목입니다. 14장에 모아두었습니다.

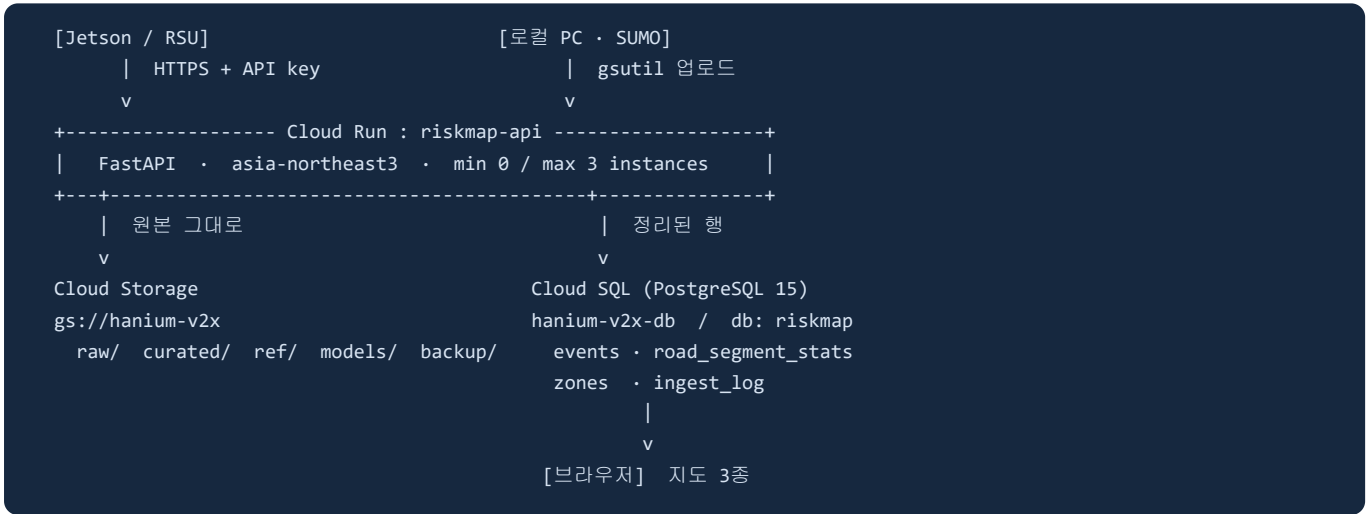
명령어의 <...>는 실제 값으로 바꿔 넣는 자리입니다.

gcloud 옵션은 버전에 따라 다를 수 있습니다. 안 되면 --help로 확인하거나 콘솔에서 하세요.

1 목표 아키텍처와 리소스 목록

이름을 먼저 확정합니다. 이름이 흔들리면 문서와 코드가 어긋납니다.

1.1 전체 구성



1.2 만들 리소스

서비스	이름 / 사양	용도	상태
Cloud Storage	gs://hanium-v2x	원본 파일 보관 (raw / curated / ref / models / backup)	있음
Cloud SQL	hanium-v2x-db PostgreSQL 15 · db-f1-micro SSD 10GB	이벤트·집계 테이블. DB명 riskmap	만들 것
Artifact Registry	hanium-v2x (docker)	Cloud Run에 올릴 컨테이너 이미지 저장소	만들 것
Cloud Run	riskmap-api 512MiB · min 0 / max 3	FastAPI 서버 + 지도 정적 파일 서빙	만들 것
Secret Manager	db-url api-key-jetson api-key-importer	DB 접속 문자열, 업로드 인증 키	만들 것
서비스 계정	riskmap-run sumo-importer	Cloud Run 실행용 / 배치 적재용	만들 것
Cloud Scheduler	refresh-stats (5분)	집계 테이블 갱신 엔드포인트 주기 호출	나중에

이름 규칙

환경 접미사(-dev, -prod)나 리전 접미사를 붙이지 않았습니다. 이 프로젝트는 환경이 하나뿐이라 단순한 이름이 낫습니다. 대신 버킷과 DB 이름을 hanium-v2x로 맞춰 한눈에 짝을 알아볼 수 있게 했습니다.

1.3 설계 결정과 근거

결정	근거
Cloud Run (VM 아님)	요청이 없으면 인스턴스가 0으로 줄어 비용이 거의 발생하지 않습니다. HTTPS 주소와 인증서가 자동으로 붙습니다. VM(Compute Engine)은 24시간 과금됩니다
max-instances 3	학생 프로젝트 트래픽에 충분하고, 무한 확장으로 인한 요금 사고를 구조적으로 막습니다
PostgreSQL (MySQL 아님)	나중에 PostGIS 확장을 켜면 "가장 가까운 도로 찾기"를 DB가 처리할 수 있습니다. 지금은 안 써도 문을 열어둡니다
단일 영역 (HA 아님)	고가용성 구성은 요금이 약 2배입니다. 서버가 잠깐 멈춰도 현장 안전 기능에 영향이 없습니다
버킷 1개 + 접두사	버킷을 여러 개 만들면 권한·수명주기를 각각 관리해야 합니다. 접두사(raw/, curated/)로 나누면 규칙을 한 곳에서 관리합니다

2 데이터 파일 카탈로그 — 무엇을 어디로

전부 올리는 게 아닙니다. 올릴 것 · 보관만 할 것 · 안 올릴 것을 먼저 나눕니다.

2.1 전체 매핑표

파일 (현재 위치)	정체	목적지	이유
Jetson: final_risk_log.csv step6/7/8_*.csv	현장 실시간 로그	GCS raw/live/ + DB events	실측 지도의 원천. 업로더가 실시간으로 전송
dataset/scenario_00X/ event_result.csv	SUMO 결과 22컬럼 시나리오 6개	GCS raw/sumo/ + DB events	시물 지도의 원천. 좌표 변환 후 적재
zones/zone_definition.csv	위험구역 4개 (Z01~Z04)	GCS ref/ + DB zones	지도 오버레이와 구역 판정에 사용
dataset/master_dataset.csv	전 시나리오 병합 24컬럼 · 약 7.9만 행 · 12.9MB	GCS raw/sumo/ 보관만	AI 학습용. 지도가 읽지 않으므로 DB에 넣지 않음
dataset/labeled_master_ dataset.csv	AI 학습 입력 23컬럼 · 약 1.26만 행	GCS raw/sumo/ 보관만	재현성 확보용. 학습을 다시 돌릴 때 필요
scenario_00X/feature.csv scenario_00X/risk_result.csv	중간 산출물 (9·11컬럼)	올리지 않음	event_result.csv에 모두 포함되어 있음
map/app/data/roads.json	OSM 도로망 117KB	GCS ref/ + 앱 이미지에 동봉	도로 스냅 기준. 서버가 시작할 때 읽으므로 이미지에 포함시키는 편이 빠름
AI_Model/.../risk_transformer.onnx	학습된 AI 모델	GCS models/ 보관만	추론은 Jetson이 수행. 서버는 사용하지 않음

굵게 표시된 3개만 DB에 들어갑니다

나머지는 버킷에 **보관만** 합니다. 나중에 필요해질 때 꺼내 쓰기 위한 것이지 지금 지도에 쓰이지 않습니다. “일단 다 DB에 넣어두자”는 생각은 비용과 복잡도만 늘립니다.

2.2 판단 기준

어떤 데이터를 파일로 둘지 표로 넣을지 헷갈릴 때는 한 문장으로 판단하세요.

“이 데이터를 지도 화면이 직접 읽는가?”

예 → Cloud SQL. 아니오 → Cloud Storage.

master_dataset.csv는 7.9만 행이지만 지도가 읽지 않으므로 DB에 넣지 않습니다. 반대로 zone_definition.csv는 4행뿐이지만 지도가 매번 읽으므로 DB에 넣습니다. 크기가 아니라 용도로 결정하세요.

2.3 컬럼 매핑 — SUMO CSV에서 DB로

event_result.csv의 22개 컬럼 중 실제로 쓰는 것은 7개입니다.

CSV 컬럼	→ DB 컬럼	변환 규칙
event_x, event_y	lng, lat	필수 변환. SUMO 로컬 미터 → 위경도 (3.4절)
risk_level	risk	0~3 그대로. 서버가 재계산하지 않습니다
ttc	ttc	9999는 “미접근” 센티넬 → NULL로 변환
distance_m	distance_m	그대로
zone_id	zone_id	Z01~Z04 또는 OUT
ts_ms	occurred_at	시물 시작 기준 밀리초 → 기준 시각 + ts_ms로 절대 시각 환산
event_id	event_uid	{scenario_id}-{event_id} 형태로 조합해 전역 고유하게
(파일 경로에서)	scenario_id	scenario_000 등
(고정)	source	'sumo'

나머지 컬럼(ped_x, veh_speed_mps, risk_score, zone_name...)은 DB에 넣지 않습니다. 필요해지면 raw/의 원본 CSV에서 다시 뽑으면 됩니다.

3 Cloud Storage 설계

버킷은 이미 있습니다. 안을 어떻게 나눌지만 정하면 됩니다.

3.1 폴더 구조

```
gs://hanium-v2x/
|
+-- raw/                                # 원본 · 절대 수정/삭제 금지
|   +-- live/dt=2026-07-26/
|       |   events-1430.jsonl          # 실시간 수신 원본 (시간별 묶음)
|   +-- sumo/scenario_000/
|       |   event_result.csv           # SUMO 결과 그대로
|       |   master_dataset.csv         # 보관용
|       |   labeled_master_dataset.csv # 보관용
|
+-- curated/                            # 가공본 · 지워도 raw에서 재생성 가능
|   +-- sumo/scenario_000/
|       |   converted-20260726.csv     # 위경도 변환 후 실제 적재한 파일
|
+-- ref/                                # 기준 데이터 · 거의 안 바뀜
|   |   zone_definition.csv
|   |   roads.json
|
+-- models/
|   |   risk_transformer.onnx          # 학습 산출물 보관
|
+-- backup/
|   |   sql/2026-07-26/riskmap.dump    # DB 수동 덤프
```

3.2 명명 규칙

규칙	이유
dt=YYYY-MM-DD	날짜를 폴더명에 넣으면 기간별로 지우거나 옮기기 쉽고, 나중에 BigQuery로 읽을 때 그대로 파티션으로 인식됩니다
events-HHMM.jsonl	시간별로 파일을 나누면 한 파일이 지나치게 커지지 않고, 문제가 생긴 시간대만 확인할 수 있습니다
converted-YYYYMMDD.csv	변환식을 고쳐 다시 만들면 날짜가 달라져 이전 것과 구분됩니다. 뺄어쓰지 마세요
시나리오 폴더로	파일명에 넣지 말고 sumo/scenario_000/처럼 폴더로 나눕니다. 시나리오 단위로 통째 처리하기 쉽습니다

3.3 수명주기 규칙

콘솔 → 버킷 hanium-v2x → 수명 주기에서 설정합니다.

대상	동작	이유
raw/	90일 후 Nearline 전환	오래된 원본은 거의 안 읽습니다. 보관 요금이 내려갑니다 (읽을 때 요금이 붙지만 드물게 읽으므로 이득)
curated/	30일 후 삭제	raw에서 다시 만들 수 있습니다. 용량 정리 1순위
backup/	180일 후 삭제	Cloud SQL 자동 백업이 별도로 있으므로 수동 덤핑은 오래 둘 필요가 없습니다
ref/, models/	규칙 없음	작고 계속 필요합니다

raw는 손대지 않습니다

append만 하고 수정·삭제하지 않습니다. **변환식이 틀렸을 때 되돌아갈 유일한 지점**입니다. 버킷 설정에서 **객체 버전 관리**를 켜두면 실수로 덮어써도 이전 버전을 복구할 수 있습니다 (저장 용량이 늘어나므로 raw만 켜는 것을 권합니다).

3.4 업로드 명령

```
# 기준 데이터 (한 번만)
gsutil cp zones/zone_definition.csv gs://hanium-v2x/ref/
gsutil cp map/app/data/roads.json gs://hanium-v2x/ref/

# SUMO 원본 6개 시나리오 (-m 은 병렬 전송)
gsutil -m cp -r dataset/scenario_00* gs://hanium-v2x/raw/sumo/

# 대용량 병합본
gsutil cp dataset/master_dataset.csv gs://hanium-v2x/raw/sumo/
gsutil cp dataset/labeled_master_dataset.csv gs://hanium-v2x/raw/sumo/

# 모델
gsutil cp AI_Model/transformer/models/risk_transformer.onnx gs://hanium-v2x/models/

# 확인
gsutil ls -r gs://hanium-v2x/
gsutil du -sh gs://hanium-v2x/
```

3.5 권한

버킷은 **공개하지 않습니다**. 현재 “공개 아님” 상태가 정답입니다. 지도에 쓰이는 데이터는 API를 통해 나가지, 버킷을 직접 열어주지 않습니다.

주체	역할	범위
riskmap-run	Storage Object Creator	실시간 원본 append. 읽기·삭제 권한 불필요
sumo-importer	Storage Object Admin	배치 업로드 및 변환본 관리
팀원 개인 계정	Storage Object Viewer	확인용 읽기. 쓰기는 스크립트로만

4 Cloud SQL 설계와 스키마

한 번 만들면 리전과 DB 종류는 바꿀 수 없습니다. 사양은 나중에 올릴 수 있습니다.

4.1 인스턴스 사양

항목	값	이유
인스턴스 ID	hanium-v2x-db	버킷 이름과 맞춰 짝을 알아보기 쉽게
DB 엔진	PostgreSQL 15	PostGIS 확장 가능성. 집계 SQL 문법이 풍부
리전	asia-northeast3	버킷-Cloud Run과 동일. 다르면 전송 요금과 지연 발생
가용성	단일 영역	HA는 요금 약 2배. 서버가 잠깐 멈춰도 현장 안전에 영향 없음
머신 유형	db-f1-micro	가장 작은 공유 코어. 느려지면 나중에 올리면 됩니다
스토리지	SSD 10GB · 자동 증가 ON	이벤트 1건이 수백 바이트라 10GB면 수천만 건. 자동 증가를 켜두면 가득 차서 멈추는 사고를 막습니다
백업	자동 백업 ON · 7일	실수로 DELETE 했을 때 되돌릴 유일한 수단
공개 IP	끄기	중요. 켜면 전 세계에서 접속 시도가 들어옵니다. Cloud SQL Auth Proxy로 접속하세요
DB / 사용자	riskmap / riskmap_app	postgres 슈퍼유저를 앱에서 직접 쓰지 않습니다

4.2 로컬에서 접속하기

공개 IP를 꺾으므로 **Cloud SQL Auth Proxy**로 접속합니다. 프록시가 인증을 대신 처리하고 로컬 포트로 터널을 열어줍니다.

```
# 프록시 실행 (터미널 하나를 계속 점유합니다)
cloud-sql-proxy hanium-ssu-kpc:asia-northeast3:hanium-v2x-db --port 5432

# 다른 터미널에서 접속 - 로컬 DB처럼 보입니다
psql "host=127.0.0.1 port=5432 dbname=riskmap user=riskmap_app"
```

4.3 테이블 ① events — 모든 위험 이벤트

```
CREATE TABLE events (
  id          BIGSERIAL PRIMARY KEY,
  event_uid   TEXT NOT NULL UNIQUE,      -- 중복 방지 (역등성 키)
  source      TEXT NOT NULL,             -- 'live' | 'sumo'
  device_id   TEXT,                      -- jetson-01 등 (live)
  scenario_id TEXT,                      -- scenario_000 (sumo)
  lat         DOUBLE PRECISION NOT NULL,
  lng         DOUBLE PRECISION NOT NULL,
  edge_id     TEXT,                      -- 스냅된 도로 구간 (저장 시 계산)
  risk        SMALLINT NOT NULL
    CHECK (risk BETWEEN 0 AND 3),
  ttc         DOUBLE PRECISION,          -- 미접근이면 NULL (9999 금지)
  distance_m  DOUBLE PRECISION,
  zone_id     TEXT,                      -- Z01~Z04 / OUT
  occurred_at TIMESTAMPTZ NOT NULL,      -- 현장에서 일어난 시각
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now() -- 서버가 받은 시각
);

CREATE INDEX ix_ev_src_time ON events (source, occurred_at);
CREATE INDEX ix_ev_edge_src ON events (edge_id, source);
CREATE INDEX ix_ev_scenario ON events (scenario_id) WHERE scenario_id IS NOT NULL;
```

핵심 컬럼

컬럼	설명
event_uid	Jetson이 네트워크 복구 후 재전송하면 같은 이벤트가 두 번 옵니다. UNIQUE 제약이 없으면 통계가 부풀려집니다. 생성 규칙 — live: {device_id}-{seq} · sumo: {scenario_id}-{event_id}
edge_id	저장할 때 한 번만 계산해 넣습니다. 조회할 때마다 도로 스냅을 다시 하면 지도가 느려집니다
occurred_at created_at	둘을 나눠야 “언제 일어났나”와 “언제 서버에 도착했나”를 구분할 수 있습니다. 오프라인 큐에 쌓였다 늦게 올라오면 두 값이 크게 벌어집니다 — 정상입니다
ttc	SUMO CSV의 9999를 그대로 넣으면 평균 TTC가 완전히 망가집니다. 반드시 NULL로

4.4 테이블 ② road_segment_stats — 지도가 읽는 표

```
CREATE TABLE road_segment_stats (
  edge_id   TEXT NOT NULL,
  source    TEXT NOT NULL,      -- 'live' | 'sumo' | 'combined'
  p1_lat    DOUBLE PRECISION, p1_lng DOUBLE PRECISION,
  p2_lat    DOUBLE PRECISION, p2_lng DOUBLE PRECISION,
  event_count INTEGER NOT NULL,
  avg_risk  DOUBLE PRECISION NOT NULL,
  grade     SMALLINT NOT NULL,  -- round(avg_risk), 0~3
  avg_ttc   DOUBLE PRECISION,
  agreement TEXT,               -- combined 행에만: both|live_only|sumo_only
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (edge_id, source)
);
```

구간 하나당 행이 3개 생깁니다 — live, sumo, combined. 지도 3종이 각각 이 행 하나씩을 읽습니다. 이벤트가 아무리 많아져도 지도가 읽는 행 수는 도로 구간 수 × 3으로 일정합니다.

4.5 테이블 ③④ zones · ingest_log

```
-- 위험구역 정의 (zone_definition.csv 를 위경도로 변환해 적재)
CREATE TABLE zones (
  zone_id    TEXT PRIMARY KEY,      -- Z01~Z04
  zone_name  TEXT, zone_type TEXT,
  center_lat DOUBLE PRECISION,
  center_lng DOUBLE PRECISION,
  radius_m   DOUBLE PRECISION,
  base_risk  SMALLINT              -- 0~5 (risk 0~3과 다른 척도임에 주의)
);

-- 어떤 파일을 언제 몇 건 넣었나
CREATE TABLE ingest_log (
  id          BIGSERIAL PRIMARY KEY,
  gcs_uri     TEXT NOT NULL UNIQUE,  -- 같은 파일 재적재 차단
  source      TEXT NOT NULL,
  scenario_id TEXT,
  row_count   INTEGER NOT NULL,
  inserted_count INTEGER NOT NULL,
  skipped_count INTEGER NOT NULL,
  status      TEXT NOT NULL,        -- ok | failed | partial
  error       TEXT,
  started_at  TIMESTAMPTZ NOT NULL,
  finished_at TIMESTAMPTZ
);
```

ingest_log가 왜 필요한가

SUMO CSV를 실수로 두 번 임포트하면 이벤트가 두 배가 됩니다. gcs_uri에 UNIQUE를 걸어두면 같은 파일을 다시 넣으려 할 때 바로 막힙니다. 문제가 생겼을 때 “언제 무엇이 몇 건 들어왔나”를 확인하는 유일한 기록이기도 합니다.

4.6 스키마 관리 — Alembic

스키마는 앞으로도 계속 바뀝니다. 손으로 ALTER TABLE을 치면 로컬과 클라우드가 곧 어긋납니다.

```
pip install alembic
alembic init migrations
# alembic.ini 의 sqlalchemy.url 을 app/config.py 값과 맞춘 뒤
alembic revision --autogenerate -m "initial schema"
alembic upgrade head
```

Alembic을 도입한 뒤에는 앱 시작 시 Base.metadata.create_all()을 호출하지 마세요. 그 함수는 이미 있는 테이블의 컬럼을 추가해주지 않아 조용히 어긋납니다.

5 지도 3종 데이터 모델

같은 events 테이블에서 지도 3개가 나옵니다.

5.1 세 지도의 정의

지도	source	보여주는 것
① 실측	live	실제 시제품이 현장에서 낸 위험 이벤트만. 건수는 적지만 신뢰도 최상 — “진짜 여기서 위험했다”
② 시뮬	sumo	SUMO가 만든 가상 위험 상황. 시나리오별 필터 — “이런 상황이면 여기가 위험해진다”
③ 종합	combined	두 지도를 겹쳐서. 등급은 max, 출처 일치 여부 표시 — “예측과 실재가 맞았는가”

5.2 집계 SQL

```
-- ① live 행 (sumo도 source만 바꿔 동일하게)
INSERT INTO road_segment_stats
    (edge_id, source, p1_lat, p1_lng, p2_lat, p2_lng,
     event_count, avg_risk, grade, avg_ttc, updated_at)
SELECT e.edge_id, 'live',
       r.p1_lat, r.p1_lng, r.p2_lat, r.p2_lng,
       COUNT(*), AVG(e.risk), ROUND(AVG(e.risk))::smallint,
       AVG(e.ttc), now()
FROM events e JOIN road_edges r ON r.edge_id = e.edge_id
WHERE e.source = 'live' AND e.edge_id IS NOT NULL
GROUP BY e.edge_id, r.p1_lat, r.p1_lng, r.p2_lat, r.p2_lng
ON CONFLICT (edge_id, source) DO UPDATE SET
    event_count = EXCLUDED.event_count,
    avg_risk    = EXCLUDED.avg_risk,
    grade       = EXCLUDED.grade,
    avg_ttc     = EXCLUDED.avg_ttc,
    updated_at  = now();
```

```
-- ③ combined 행 : events를 다시 읽지 않고 ①의 결과만 본다
INSERT INTO road_segment_stats (edge_id, source, ..., grade, agreement, updated_at)
SELECT
    COALESCE(l.edge_id, s.edge_id), 'combined', ...,
    GREATEST(COALESCE(l.grade, 0), COALESCE(s.grade, 0)),
    CASE
        WHEN l.grade >= 1 AND s.grade >= 1 THEN 'both'
        WHEN l.grade >= 1 THEN 'live_only'
        ELSE 'sumo_only'
    END, now()
FROM (SELECT * FROM road_segment_stats WHERE source='live') l
FULL JOIN (SELECT * FROM road_segment_stats WHERE source='sumo') s
    ON l.edge_id = s.edge_id
ON CONFLICT (edge_id, source) DO UPDATE SET ...;
```

합산이 아니라 GREATEST(max)인 이유

이벤트를 그냥 합쳐 평균 내면 SUMO 수만 건에 실측 몇 건이 묻혀버립니다. 종합 지도가 사실상 시물 지도와 같아집니다.

소스별로 등급을 따로 낸 뒤 더 높은 쪽을 취하면 건수 불균형이 사라지고, 이 시스템이 이미 쓰는 “안전 우선 max 융합” 원칙 ($\text{final} = \max(\text{rule}, \text{predicted}, \text{zone}, \text{fail_safe})$)과 같은 방식이 됩니다.

5.3 agreement가 이 지도의 진짜 가치

값	뜻	이 값이 나오면
both	두 소스 모두 위험(등급 1↑)이라고 봄	가장 확실한 위험 구간. 발표에서 강조할 지점
live_only	실측에서만 위험	시물이 놓친 상황 → SUMO 시나리오에 추가해야 함
sumo_only	시물에서만 위험	아직 실측이 없는 구간 → 다음 측정 대상

일치율(both 구간 / 전체 위험 구간)은 “시뮬레이션이 현실을 얼마나 잘 맞췄는가”를 한 숫자로 보여주며, 발표·논문 등에 그대로 인용할 수 있습니다.

5.4 API

```
GET /api/segments?source=live
GET /api/segments?source=sumo&scenario=scenario_003
GET /api/segments?source=combined&from=2026-07-01&to=2026-07-26

# 응답
[ { "edge_id": "w123-4",
  "points": [[37.4997,126.9516],[37.4999,126.9518]],
  "grade": 3, "events": 12, "avg": 2.83, "ttc": 0.75,
  "agreement": "both" } ]      // combined 일 때만
```

지도 3개를 API 3개로 만들지 마세요. **엔드포인트 하나에 파라미터로 분기**하면 유지보수가 1/3이 됩니다.

6 적재 파이프라인

데이터가 클라우드로 들어오는 두 갈래를 각각 설계합니다.

6.1 경로 A — 실시간 (Jetson → 클라우드)

1. Jetson 위험 판정 발생 → event_uid 생성 → 로컬 JSONL에
2. 업로더 스레드 파일을 읽어 순서대로 POST, 실패 시 지수 백오프 재시도
3. Cloud Run API 키 검증 → 좌표 범위 검사 → edge_id 계산 → INSERT
4. 저장 Cloud SQL 1행 + GCS raw/live/ 에 원본 JSON 한 줄 append

요청 형식

```
POST /api/events      Header: X-API-Key: <api-key-jetson>
{
  "event_uid":  "jetson-01-000482",
  "source":     "live",
  "device_id":  "jetson-01",
  "lat": 37.4997, "lng": 126.9516,
  "risk": 3, "ttc": 0.8, "distance_m": 2.4,
  "zone_id": "Z04",
  "occurred_at": "2026-07-26T14:30:12+09:00"
}
```

서버 측 저장 (역등)

```
from sqlalchemy.dialects.postgresql import insert

stmt = (insert(Event).values(**payload)
        .on_conflict_do_nothing(index_elements=["event_uid"]))
result = db.execute(stmt)
db.commit()
# result.rowcount == 0 이면 이미 있던 이벤트 → 200 OK로 응답 (에러 아님)
```

끊겨도 잃지 않는 구조 — 파일 큐

현장 네트워크는 반드시 끊깁니다. 핵심은 “보내기 전에 먼저 파일에 쓴다”입니다. 메모리 큐만 쓰면 프로세스가 죽는 순간 사라집니다.

기존 03_jetson/cloud_uploader.py에 큐 + 백그라운드 워커 구조가 있지만 **메모리 큐**입니다. 파일 기반으로 바꾸는 것이 이번 작업입니다. 전송에 성공한 줄만 표시해두고, 재시작하면 표시 안 된 줄부터 이어서 보냅니다.

업로드 트리거 정책 [팀 확인 필요]

- 모든 판정을 올리면 약 10 Hz × 계속 = 데이터가 폭증합니다.
- 권장: **risk ≥ 1인 경우만**, 그리고 **같은 위치-같은 등급이 연속되면 묶어서 1건**(중복 억제).
- 정책을 정한 뒤 반드시 문서에 적어두세요. 나중에 “왜 데이터가 이것뿐이지?”라는 질문이 반드시 나옵니다.

6.2 경로 B — 배치 (SUMO → 클라우드)

수만 건을 한 건씩 POST하면 몇 시간이 걸립니다. **파일로 올리고 서버가 읽게** 하세요.

```
# 1) 원본 그대로 올리기
gsutil -m cp dataset/scenario_000/event_result.csv \
gs://hanium-v2x/raw/sumo/scenario_000/

# 2) 좌표 변환 후 올리기
python convert_sumo.py scenario_000          # → converted-20260726.csv
gsutil cp converted-20260726.csv \
gs://hanium-v2x/curated/sumo/scenario_000/

# 3) 서버에 적재 요청 (서버가 GCS에서 읽어 500건씩 나눠 INSERT)
curl -X POST $URL/api/import -H "X-API-Key: $IMPORTER_KEY" \
-H "Content-Type: application/json" \
-d '{"gcs_uri":"gs://hanium-v2x/curated/sumo/scenario_000/converted-20260726.csv",
"source":"sumo","scenario_id":"scenario_000"}'
```

변환 스크립트의 핵심

```
import sumolib, csv, datetime

net = sumolib.net.readNet("campus.net.xml")
BASE = datetime.datetime(2026, 7, 8, 9, 0, tzinfo=KST) # 시를 기준 시각

for row in csv.DictReader(open(src)):
    lng, lat = net.convertXY2LonLat(float(row["event_x"]),
                                     float(row["event_y"])) # 반환 순서 주의
    if not (37.4 <= lat <= 37.6 and 126.8 <= lng <= 127.1):
        raise ValueError(f"좌표 범위 이탈: {lat},{lng}") # 조용히 넘기지 말 것

    ttc = float(row["ttc"])
    out = {
        "event_uid": f"{scenario}-{row['event_id']}",
        "source": "sumo",
        "scenario_id": scenario,
        "lat": lat, "lng": lng,
        "risk": int(row["risk_level"]),
        "ttc": None if ttc >= 9999 else ttc, # 센티널 → NULL
        "distance_m": float(row["distance_m"]),
        "zone_id": row["zone_id"],
        "occurred_at": BASE + datetime.timedelta(milliseconds=int(row["ts_ms"])),
    }
```

업로드 전 반드시 검산

변환한 좌표가 **위도 37.4~37.6 / 경도 126.8~127.1** 범위인지 확인하세요. 벗어나면 변환식이 틀린 것입니다. **먼저 10건만 올려 지도에서 눈으로 확인한 뒤 전체를 올리세요.**

[팀 확인 필요] net.xml이 실제 지역에 지오레퍼런싱되어 있어야 이 변환이 유효합니다. 안 되어 있으면 기준점 오프셋 방식을 팀과 합의해야 합니다.

6.3 집계 갱신

이벤트가 들어올 때마다 갱신할 수도 있고, 주기적으로 전체를 다시 계산할 수도 있습니다. 데이터 규모가 크지 않으므로 **주기적 재계산이 단순하고 안전합니다.**

갱신 엔드포인트를 만들고 Cloud Scheduler가 5분마다 호출

POST /api/admin/refresh-stats Header: X-API-Key: <api-key-importer>

```
gcloud scheduler jobs create http refresh-stats \  
  --schedule="*/5 * * * *" --location=asia-northeast3 \  
  --uri="$URL/api/admin/refresh-stats" --http-method=POST \  
  --headers="X-API-Key=<키>"
```


7 Cloud Run 배포

컨테이너 하나로 API와 지도 화면을 함께 서빙합니다.

7.1 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# Cloud Run은 PORT 환경변수로 포트를 알려줍니다. 8000 하드코딩 금지
CMD exec uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

7.2 requirements.txt

```
fastapi
uvicorn[standard]
sqlalchemy
psycopg[binary]          # PostgreSQL 드라이버
pydantic
alembic
google-cloud-storage     # GCS 읽기/쓰기
cloud-sql-python-connector[pg8000] # (선택) 프록시 없이 접속할 때
```

7.3 빌드와 배포

```
# 1) 이미지 저장소 (한 번만)
gcloud artifacts repositories create hanium-v2x \
  --repository-format=docker --location=asia-northeast3

# 2) 소스에서 바로 빌드 + 푸시
gcloud builds submit \
  --tag asia-northeast3-docker.pkg.dev/hanium-ssu-kpc/hanium-v2x/riskmap:latest

# 3) 배포
gcloud run deploy riskmap-api \
  --region asia-northeast3 \
  --image asia-northeast3-docker.pkg.dev/hanium-ssu-kpc/hanium-v2x/riskmap:latest \
  --allow-unauthenticated \
  --max-instances 3 --min-instances 0 --memory 512Mi \
  --add-cloudsql-instances hanium-ssu-kpc:asia-northeast3:hanium-v2x-db \
  --service-account riskmap-run@hanium-ssu-kpc.iam.gserviceaccount.com \
  --set-secrets DATABASE_URL=db-url:latest,API_KEY=api-key-jetson:latest \
  --set-env-vars GCS_BUCKET=hanium-v2x
```

7.4 설정 항목 정리

설정	값	주의
인증	--allow-unauthenticated	지도를 누구나 봐야 하므로 필요. 대신 쓰기 API는 API 키로 차단
Cloud SQL 연결	--add-cloudsql-instances	이걸 빼면 DB 접속 실패로 500 에러가 납니다. 가장 흔한 실수
비밀값	--set-secrets	--set-env-vars로 비밀번호를 넣지 마세요. 콘솔에 그대로 보입니다
최대 인스턴스	--max-instances 3	요금 사고 방지. 나중에 필요하면 올리면 됩니다
DB 접속 문자열	?host=/cloudsql/...	Cloud Run에서는 유닉스 소켓으로 붙습니다. 로컬(프록시)과 형식이 다릅니다

DATABASE_URL 두 가지 형태

```
# 로컬 (Auth Proxy 사용)
postgresql+psycpg://riskmap_app:<PW>@127.0.0.1:5432/riskmap

# Cloud Run (유닉스 소켓)
postgresql+psycpg://riskmap_app:<PW>@/riskmap?host=/cloudsql/hanium-ssu-kpc:asia-northeast3:hanium-v2x-db
```

7.5 CORS

지도 페이지를 Cloud Run이 함께 서빙하면 CORS 문제가 없습니다. 프론트를 별도 도메인에 올릴 때만 필요하며, 그때도 **와일드카드 대신 실제 도메인만** 넣으세요.

```
app.add_middleware(CORSMiddleware,
    allow_origins=["https://riskmap-api-xxxx.a.run.app"], # "*" 금지
    allow_methods=["GET", "POST"], allow_headers=["X-API-Key", "Content-Type"])
```

8 IAM · 시크릿 · 보안

“일단 다 열고 나중에 좁히기”는 대부분 나중에 오지 않습니다.

8.1 서비스 계정

계정	역할(Role)	쓰는 곳
riskmap-run	roles/cloudsql.client roles/storage.objectCreator roles/secretmanager.secretAccessor	Cloud Run 서비스가 사용. DB 접속, 원본 append, 시크릿 읽기만 가능
sumo-importer	roles/storage.objectAdmin roles/cloudsql.client	로컬에서 배치 적재 스크립트를 돌릴 때 사용

```
gcloud iam service-accounts create riskmap-run \
  --display-name="Cloud Run riskmap service"

gcloud projects add-iam-policy-binding hanium-ssu-kpc \
  --member=serviceAccount:riskmap-run@hanium-ssu-kpc.iam.gserviceaccount.com \
  --role=roles/cloudsql.client

# 버킷 단위로만 권한 주기 (프로젝트 전체보다 좁음)
gsutil iam ch \
  serviceAccount:riskmap-run@hanium-ssu-kpc.iam.gserviceaccount.com:objectCreator \
  gs://hanium-v2x
```

기본 서비스 계정을 쓰지 마세요

Cloud Run은 지정하지 않으면 **Compute Engine 기본 서비스 계정**을 씁니다. 이 계정은 프로젝트 편집자 권한을 가진 경우가 많아, 앱이 뚫리면 프로젝트 전체가 노출됩니다. 반드시 `--service-account`로 전용 계정을 지정하세요.

8.2 시크릿

이름	내용
db-url	DB 접속 문자열 (비밀번호 포함)
api-key-jetson	현장 장치가 이벤트를 올릴 때 쓰는 키
api-key-importer	배치 적재·집계 갱신용 키. 분리해야 유출 시 하나만 교체할 수 있습니다

```
# 파일로 남기지 말고 표준입력으로 생성
printf '%s' "postgresql+psycopg://riskmap_app:...@/riskmap?host=/cloudsql/..." \
  | gcloud secrets create db-url --data-file=-

# 랜덤 키 생성 (파이썬)
python -c "import secrets; print(secrets.token_urlsafe(32))" \
  | tr -d '\n' | gcloud secrets create api-key-jetson --data-file=-

# 값 교체 (새 버전 추가)
printf '%s' "<새 키>" | gcloud secrets versions add api-key-jetson --data-file=-
```

8.3 API 인증 구현

```
from fastapi import Header, HTTPException, Depends
import os, hmac

def require_api_key(x_api_key: str = Header(default="")):
    expected = os.environ["API_KEY"]
    # == 비교 대신 compare_digest - 타이밍 공격 방지
    if not hmac.compare_digest(x_api_key, expected):
        raise HTTPException(status_code=401, detail="invalid api key")

@app.post("/api/events", dependencies=[Depends(require_api_key)])
def upload_event(...): ...
```

8.4 공개 범위 정리

엔드포인트	접근	이유
GET / (지도 페이지)	공개	발표·심사에서 누구나 봐야 함
GET /api/segments	공개	지도가 읽는 집계 데이터. 개인정보 없음
POST /api/events	API 키	막지 않으면 누구나 가짜 데이터를 넣을 수 있음
POST /api/import	API 키	대량 적재. 반드시 차단
POST /api/admin/*	API 키	집계 갱신 등 관리 기능
버킷 직접 접근	비공개	현재 상태 유지. 데이터는 API로만 나갑니다

키가 Git에 올라갔다면

커밋을 지워도 이미 유출된 것으로 간주하고 즉시 교체하세요. .gitignore에 *.json(서비스 계정 키), .env를 반드시 넣으세요.

9 실행 순서

이 순서대로 하세요. 앞의 4개는 오늘 안에 끝냅니다.

콘솔 먼저, CLI는 나중에

처음 한 번은 콘솔(웹 UI)에서 눈으로 보면서 만드는 것을 권합니다. 무엇을 만들고 있는지 감이 잡힙니다. 익숙해진 뒤 CLI로 바꾸면 됩니다. 결과는 같습니다.

1. 예산 알림 확인 ★

콘솔: 결제 → 예산 및 알림

월 한도와 알림 임계값(50 / 90 / 100 %)을 설정합니다. 이미 되어 있으면 건너뜁니다. 예산 알림은 과금을 막아주지는 않지만, 메일을 받으면 바로 끌 수 있습니다.

완료 기준 — 예산 설정 확인 메일을 받았다.

2. API 사용 설정

콘솔: API 및 서비스 → 라이브러리

Cloud SQL Admin · Cloud Run · Artifact Registry · Secret Manager · Cloud Build를 켭니다.

```
gcloud services enable sqladmin.googleapis.com run.googleapis.com \
  artifactregistry.googleapis.com secretmanager.googleapis.com \
  cloudbuild.googleapis.com storage.googleapis.com
```

완료 기준 — `gcloud services list --enabled`에 위 항목이 보인다.

3. 버킷 폴더 만들고 기준 파일 올리기

콘솔: Cloud Storage → hanium-v2x

GCS에는 진짜 폴더가 없고 접두사만 있습니다. 파일을 올리면 폴더가 자동으로 생깁니다.

```
gsutil cp zones/zone_definition.csv gs://hanium-v2x/ref/
gsutil cp map/app/data/roads.json gs://hanium-v2x/ref/
gsutil -m cp -r dataset/scenario_00* gs://hanium-v2x/raw/sumo/
gsutil ls -r gs://hanium-v2x/
```

완료 기준 — `gsutil ls`로 `ref/` 와 `raw/sumo/` 아래 파일이 보인다.

4. 수명주기 규칙 설정

콘솔: 버킷 → 수명 주기 → 규칙 추가

`raw/` 90일 Nearline 전환, `curated/` 30일 삭제 (3.3절 표 참고).

완료 기준 — 수명 주기 탭에 규칙 2개가 보인다.

5. Cloud SQL 인스턴스 생성 ★ 되돌리기 어려움

콘솔: SQL → 인스턴스 만들기 → PostgreSQL

4.1절 표의 값을 그대로 입력합니다. 리전과 DB 엔진은 나중에 못 바꿉니다. 생성에 5~10분 걸립니다.

```
gcloud sql instances create hanium-v2x-db \
  --database-version=POSTGRES_15 --tier=db-f1-micro \
  --region=asia-northeast3 --storage-size=10GB \
  --storage-auto-increase --backup --no-assign-ip
```

완료 기준 — 인스턴스 상태가 “실행 중”이고 공개 IP가 없다.

6. DB와 사용자 생성

```
gcloud sql databases create riskmap --instance=hanium-v2x-db
gcloud sql users create riskmap_app --instance=hanium-v2x-db --password=<강한 비밀번호>
```

완료 기준 — 인스턴스 상세의 데이터베이스 탭에 riskmap이 보인다.

7. 테이블 생성 ★

Cloud SQL Studio(콘솔) 또는 Auth Proxy + psql

4장의 DDL 4개(events, road_segment_stats, zones, ingest_log)를 실행합니다. 가능하면 Alembic 마이그레이션으로 만드세요 — 앞으로 스키마가 계속 바뀝니다.

완료 기준 — \dt에 테이블 4개가 보이고, 같은 event_uid를 두 번 INSERT하면 두 번째가 막힌다.

8. 시크릿과 서비스 계정

8장의 명령으로 시크릿 3개와 서비스 계정 2개를 만들고 권한을 부여합니다.

완료 기준 — gcloud secrets list에 3개가 보인다.

9. Cloud Run 배포 ★ 분기점

7장의 Dockerfile을 만들고 빌드·배포합니다. 이 단계가 끝나면 고정 HTTPS 주소가 생깁니다.

완료 기준 — 발급된 https://...run.app에서 /docs가 열리고, DB 조회 API가 500 없이 응답한다.

10. 데이터 적재 및 지도 확인

SUMO 시나리오 1개를 변환해 10건만 먼저 올리고 지도에서 확인 → 문제없으면 전체 적재 → Jetson 업로더 연결 → 집계 갱신 → 지도 3중 확인.

완료 기준 — 실측·시물·종합 세 탭이 모두 데이터를 표시한다.

10 운영 — 비용 · 백업 · 모니터링

만들고 나면 신경 쓸 것은 세 가지뿐입니다.

10.1 비용

금액은 직접 확인하세요

요금 체계가 자주 바뀌므로 이 문서에 금액을 적지 않습니다. **GCP 요금 계산기**에서 아래 조건으로 계산하고, **학생 무료 크레딧** 적용 여부를 먼저 확인하세요.

항목	과금 방식	절감 방법
Cloud SQL	켜져 있는 시간에 비례	비용의 대부분이 여기서 나옵니다. 개발 중에는 밤에 인스턴스를 정지시켜도 됩니다 (데이터는 유지)
Cloud Run	요청 처리 시간에 비례	min-instances 0이면 요청이 없을 때 과금이 거의 없습니다
Cloud Storage	저장 용량 + 전송량	수 GB 수준이라 미미. 수명주기로 오래된 것을 Nearline 전환
Artifact Registry	이미지 저장 용량	오래된 이미지 태그를 정리하세요

```
# 개발 중 밤에 DB 정지 / 아침에 시작
gcloud sql instances patch hanium-v2x-db --activation-policy=NEVER # 정지
gcloud sql instances patch hanium-v2x-db --activation-policy=ALWAYS # 시작
```

10.2 백업

대상	방법	주기
Cloud SQL	자동 백업	매일 · 7일 보관 (인스턴스 생성 시 설정)
Cloud SQL 수동 덤프	<code>gcloud sql export sql</code>	월 1회 · <code>gs://hanium-v2x/backup/sql/</code> 로
GCS raw/	객체 버전 관리	상시 (덮어쓰기 사고 대비)

```
gcloud sql export sql hanium-v2x-db \
gs://hanium-v2x/backup/sql/2026-07-26/riskmap.dump --database=riskmap
```

집계 테이블은 백업하지 않아도 됩니다 — events에서 언제든지 다시 만들 수 있습니다. 반대로 **events 원본은 반드시 백업**해야 합니다. 실측 데이터는 다시 만들 수 없습니다.

10.3 모니터링

- **Cloud Run 로그** — 콘솔 → Cloud Run → 서비스 → 로그. 5xx 에러가 있는지 확인합니다.
- **“마지막 실측 수신 시각”** — 지도 대시보드에 띄워두세요. 이 값이 멈추면 Jetson 쪽 문제입니다. 가장 실용적인 장애 감지 방법입니다.
- **ingest_log** — 배치 적재가 실패했는지 `status='failed'`로 확인합니다.
- **예산 알림** — 요금이 예산을 넘으면 메일이 옵니다. 알림이 오면 먼저 Cloud SQL을 확인하세요.

11 완료 기준 체크리스트

각 단계가 “끝났다”고 말할 수 있는 조건입니다.

인프라

- ☐ 예산 알림이 설정되어 있고 확인 메일을 받았다
- ☐ 필요한 API가 모두 사용 설정되어 있다
- ☐ 버킷에 raw / curated / ref / models / backup 접두사가 만들어져 있다
- ☐ ref/에 zone_definition.csv, roads.json이 있다
- ☐ raw/sumo/에 시나리오 6개의 event_result.csv가 있다
- ☐ 수명주기 규칙 2개가 설정되어 있다
- ☐ Cloud SQL 인스턴스가 실행 중이고 **공개 IP가 없다**
- ☐ DB riskmap과 사용자 riskmap_app이 있다
- ☐ 테이블 4개(events, road_segment_stats, zones, ingest_log)가 있다
- ☐ Artifact Registry 저장소가 있다

보안

- ☐ 시크릿 3개가 Secret Manager에 있다
- ☐ DB 비밀번호와 API 키가 코드·Git에 없다
- ☐ 서비스 계정 2개가 있고 기본 계정을 쓰지 않는다
- ☐ 버킷이 여전히 비공개다
- ☐ POST /api/events가 API 키 없이는 401을 반환한다

데이터

- ☐ 같은 event_uid를 두 번 POST해도 DB에는 1건만 있다
- ☐ 변환된 SUMO 좌표가 위도 37.4~37.6 / 경도 126.8~127.1 범위에 있다
- ☐ ttc = 9999가 DB에 하나도 없다 (NULL로 변환되었다)
- ☐ 같은 파일을 두 번 임포트하면 ingest_log에서 막힌다
- ☐ events에 source가 live인 행과 sumo인 행이 모두 있다
- ☐ Jetson을 껐다 켜도 그 사이 이벤트가 유실 없이 올라온다

서비스

- ☐ Cloud Run 고정 https 주소로 지도가 열린다
- ☐ 팀원의 다른 기기·휴대폰에서도 접속된다
- ☐ 실측 지도 탭이 데이터를 표시한다 (source=live)
- ☐ 시물 지도 탭이 동작하고 시나리오 필터가 된다 (source=sumo)
- ☐ 종합 지도 탭에서 grade = max(live, sumo)로 계산된다
- ☐ 종합 지도에서 both / live_only / sumo_only가 구분되어 보인다

- ☐ 이벤트 5만 건 상태에서 지도가 3초 안에 뜬다
- ☐ 대시보드에 마지막 실측 수신 시각이 표시된다
- ☐ README에 최종 API 명세와 배포 명령이 적혀 있다

12 자주 막히는 곳과 해결

먼저 여기를 확인하고, 없으면 팀에 물어보세요.

증상	원인 후보	해결
Cloud Run 배포는 됐는데 모든 요청이 500	Cloud SQL 연결 미설정, 또는 DATABASE_URL 형식 오류	--add-cloudsql-instances 확인. Cloud Run에서는 ?host=/cloudsql/... 형식이어야 합니다
컨테이너가 시작하자마자 죽음	PORT 환경변수를 안 씀 (8000 하드코딩)	CMD에 --port \$PORT 사용
Secret 접근 권한 오류	서비스 계정에 secretAccessor 역할 없음	roles/secretmanager.secretAccessor 부여
업로드는 200인데 지도에 안 보임	좌표가 지도 범위 밖 (SUMO 좌표 그대로, 또는 가짜 GPS 37.0/127.0)	DB에서 lat/lng를 직접 조회해 범위 확인. 저장 전 범위 검사를 넣으세요
이벤트 건수가 실제보다 많음	재전송 또는 재임포트로 중복 저장	event_uid UNIQUE와 ingest_log가 실제로 동작하는지 확인
평균 TTC가 수천	ttc=9999 센티널을 그대로 저장	임пор터에서 NULL 변환. 이미 들어간 값은 UPDATE events SET ttc=NULL WHERE ttc>=9999
지도가 아주 느림	요청마다 전체 이벤트를 훑어 집계	road_segment_stats를 읽도록 전환. 임시로는 기간 필터로 범위 축소
도로가 아닌 엉뚱한 곳에 색칠됨	roads.json이 해당 지역 도로망이 아님	Overpass API로 실제 측정 지역 도로망을 다시 받아 교체
gsutil이 권한 오류	로그인 계정에 버킷 권한 없음	gcloud auth login 후 gcloud config get project로 프로젝트 확인
Cloud SQL 접속이 안 됨	공개 IP를 썼는데 프록시를 안 띄움	Auth Proxy를 실행한 뒤 127.0.0.1로 접속
요금이 예상보다 많이 나옴	Cloud SQL이 계속 켜져 있음	결제 → 보고서에서 항목별 확인. 개발 중엔 밤에 정지

13 용어집

클라우드 콘솔에서 자주 마주치는 말들입니다.

용어	뜻
프로젝트 (Project)	GCP의 최상위 그릇. 모든 리소스·권한·결제가 프로젝트 단위로 묶입니다. 우리는 <code>hanium-ssu-kpc</code> 하나만 씁니다
리전 (Region)	데이터센터가 있는 지역. <code>asia-northeast3</code> 은 서울. 리전이 다르면 전송 요금과 지연이 생깁니다
버킷 (Bucket)	Cloud Storage의 저장 공간 단위. 이름은 전 세계에서 유일해야 합니다
객체 (Object)	버킷에 저장된 파일 하나. GCS에는 진짜 폴더가 없고 <code>raw/live/</code> 같은 이름 접두사만 있습니다
스토리지 클래스	Standard(자주 읽음) / Nearline(월 1회) / Coldline / Archive. 뒤로 갈수록 보관은 싸고 읽기는 비쌉니다
수명주기 (Lifecycle)	“N일 지나면 클래스 변경 또는 삭제” 같은 자동 규칙
Cloud Run	컨테이너를 올리면 HTTPS 주소를 주고, 요청이 있을 때만 실행해주는 서비스. 서버를 직접 관리하지 않습니다
컨테이너 / 이미지	앱과 실행 환경을 통째로 묶은 것. Dockerfile로 만들고 Artifact Registry에 보관합니다
인스턴스 (Cloud Run)	실제로 떠 있는 앱 복제본. min 0이면 요청이 없을 때 0개까지 줄어듭니다
Cloud SQL	구글이 관리해주는 관계형 DB. 백업·패치를 대신 해줍니다
Auth Proxy	공개 IP 없이 Cloud SQL에 안전하게 접속하게 해주는 프로그램. 로컬 포트로 터널을 열어줍니다
서비스 계정	사람이 아닌 프로그램용 계정. 앱이 GCP 리소스에 접근할 때 씁니다
IAM / 역할(Role)	“누가 무엇을 할 수 있는가”를 정하는 권한 체계. 역할은 권한 묶음입니다
Secret Manager	비밀번호·API 키를 안전하게 보관하고 앱에 주입해주는 금고
멥등성 (Idempotency)	같은 요청을 여러 번 보내도 결과가 한 번 보낸 것과 같은 성질. 재전송 중복을 막는 핵심 개념
엣지 (edge, 도로)	지도에서 도로를 인접한 두 점씩 자른 구간 단위. 집계的基本 단위입니다
TTC	Time To Collision. 충돌까지 남은 예상 시간(초). 9999는 “접근하지 않음” 센티넬

14 지금 팀이 결정해야 할 것

혼자 정하면 나중에 전부 다시 해야 하는 항목들입니다.

항목	물어볼 사람	내용
SUMO 좌표 변환식	AI-데이터 담당	net.xml이 실제 지역에 지오레퍼런싱되어 있는지. 안 되어 있으면 기준점 오프셋 방식을 합의해야 합니다. 이것이 정해지지 않으면 시물 지도를 만들 수 없습니다
실측 GPS 지역	통신.하드웨어 담당	현재 실시간 파이프라인이 가짜 좌표(37.0 / 127.0)를 쓰고 있습니다. 실제 측정 장소가 확정되어야 지도 중심과 도로망(roads.json)이 정해집니다
업로드 트리거 정책	통신 담당	모든 판정을 올리면 10 Hz로 폭증합니다. risk ≥ 1만 올릴지, 중복을 어떻게 억제할지
시물 기준 시각	AI-데이터 담당	SUMO의 ts_ms는 시각 기준 상대 시간입니다. 절대 시각으로 바꾸려면 기준 시각을 정해야 합니다
저장소 위치	팀 전체	서버 코드를 정본 저장소 안 server/에 둘지 별도 저장소로 둘지. 정본 안에 두는 쪽을 권합니다 — Jetson 업로더와 API 스펙이 같이 있어야 어긋나지 않습니다

막히면 바로 물어보세요

클라우드 는 “한 줄 설정”이 안 맞아서 하루를 날리는 일이 흔합니다. 특히 IAM 권한과 Cloud SQL 연결은 에러 메시지만으로 원인을 찾기 어렵습니다. **혼자 30분 이상 막히면 화면을 캡처해 팀에 공유하세요.**

이 문서에 없는 내용을 새로 알아냈다면, 그 자리에서 문서에 추가해 두세요. 다음 사람이 같은 곳에서 막히지 않습니다.

AI 기반 V2X 협력형 지능형 시각장애인 보행 안전 지팡이 시스템 · 팀 하이퍼-송 · 2026 ICT 한이음 드림업

본 지침서는 2026년 7월 26일 기준 GCP 콘솔 상태와 저장소 실제 코드를 근거로 작성되었습니다. 리소스 이름이나 스키마를 바꾸면 이 문서도 함께 갱신하세요.